

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#_06_)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B__

Date performed: _30th September _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task code is given we have to find the error and fixing it firstly in main s0 is assigned as 2500 after that in procedure label so becomes 3 and for printing the output li a7, 1 so that values go to a0 but here is wrong calling procedure. To resolve these issues we use a temporary register t2 instead of using s0 to preserve s0 value.

Code:

Code1

```
.text

main:

li a0, 1

li s0, 2500

jal ra, procedure # call procedure

add t1, s0, a0    # result should be 1003+2500


li a7,1  #print integers

mv a0, t1      # data to display in a1

ecall    # displays result on console

li a7, 93

li a0, 0

ecall

procedure:

li t2, 3

li t0, 1000

add a0, t2, t0 #a0=3+1003
```

jalr zero, ra, 0 # return

```

1 .text
2 main:
3 li a0, 1
4 li s0, 2500
5 jal ra, procedure      # call procedure
6 add t1, s0, a0          # result should be 1003+2500
7
8 li a7,1                #print integers
9 mv a0, t1              # data to display in a1
10 ecall                 # displays result on console
11 li a7, 93
12 li a0, 0
13 ecall
14 procedure:
15 li t2, 3
16 li t0, 1000
17 add a0, t2, t0 #a0=3+1003
18 jalr zero, ra, 0      # return
19

```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00100513	addi x10 x0 1	li a0, 1
0x4	0x00001437	lui x8 1	li s0, 2500
0x8	0x9C440413	addi x8 x8 -1596	li s0, 2500
0xc	0x020000EF	jal x1 32	jal ra, procedure # call procedure
0x10	0x00A40333	add x6 x8 x10	add t1, s0, a0 # result should be 1003+2500
0x14	0x00100893	addi x17 x0 1	li a7,1 #print integers

0x18	0x00030513	addi x10 x6 0	mv a0, t1 # data to display in a1
0x1c	0x00000073	ecall	ecall # displays result on console
0x20	0x05D00893	addi x17 x0 93	li a7, 93
0x24	0x00000513	addi x10 x0 0	li a0, 0
0x28	0x00000073	ecall	ecall
0x2c	0x00300393	addi x7 x0 3	li t2, 3
0x30	0x3E800293	addi x5 x0 1000	li t0, 1000

0x34	0x00538533	add x10 x7 x5	add a0, t2, t0 #a0=3+1003
0x38	0x00008067	jalr x0 x1 0	jalr zero, ra, 0 # return

t0
(x5)

1000

t1
(x6)

3503

t2
(x7)

3

s0
(x8)

2500

Task 2:

Description:

This task is for understanding the push and pop functions of stack in memory. Firstly we assigned values to the registers then for pushing we allocate the space in memory by -12 because there are 3 registers so

3*4=12 then stored the values at offset 0 4 8 then when we want to pop it so we +12 bytes to deallocate the memory.

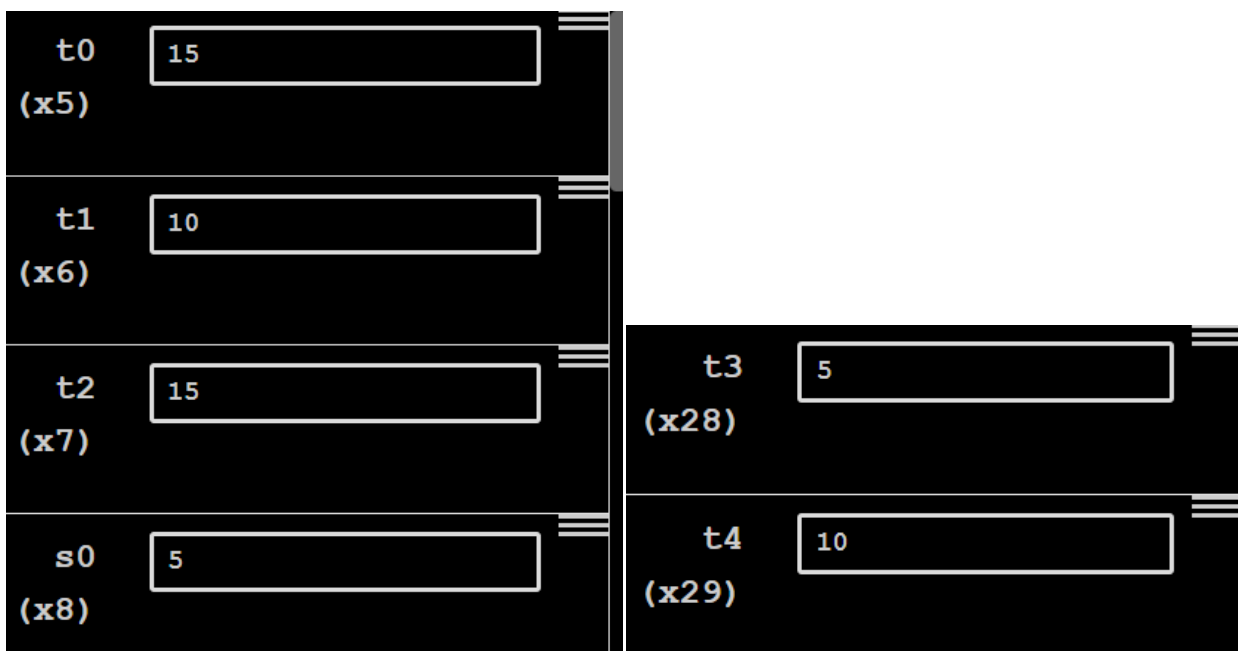
Code:

```
1 .data
2 .text
3 .globl main
4
5 addi s0, zero, 5
6 addi t1, zero, 10
7 addi t0, zero, 15
8 addi sp, sp, -12
9 sw s0, 0(sp)
10 sw t0, 4(sp)
11 sw t1, 8(sp)
12 lw t4, 8(sp)
13 lw t2, 4(sp)
14 lw t3, 0(sp)
15 addi sp, sp, 12
16
17
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00500413	addi x8 x0 5	addi s0, zero, 5
0x4	0x00A00313	addi x6 x0 10	addi t1, zero, 10
0x8	0x00F00293	addi x5 x0 15	addi t0, zero, 15
0xc	0xFF410113	addi x2 x2 -12	addi sp, sp, -12
0x10	0x00812023	sw x8 0(x2)	sw s0, 0(sp)
0x14	0x00512223	sw x5 4(x2)	sw t0, 4(sp)
0x18	0x00612423	sw x6 8(x2)	sw t1, 8(sp)

0x1c	0x00812E83	lw x29 8(x2)	lw t4, 8(sp)
0x20	0x00412383	lw x7 4(x2)	lw t2, 4(sp)
0x24	0x00012E03	lw x28 0(x2)	lw t3, 0(sp)
0x28	0x00C10113	addi x2 x2 12	addi sp, sp, 12



Task 3:

Description:

In this task a function is given eqsolver that is called in main. firstly we initialize a and b with some values let 3,2 then the input arguments are taken in a0 and a1 so we call that function by jal command that jump to eqsolver function and also keep track of return part by ra , so when we jump to eqsolver there we perform the operation that is given then result is stored in a0 and then we write jalr command that will return to the main where we store the result in t2 that is x.

Code:

```

.data
a: .word 3
b: .word 2

```

x: .word 0

.text

.globl main

main:

la t0, a

lw a0, 0(t0)

la t1, b

lw a1, 0(t1)

jal ra, EqSolver

la t2, x

sw a0, 0(t2)

li a7, 10

ecall

EqSolver:

add t3, a0, a1

sub t4, a0, a1

mul a0, t3, t4

jalr zero, ra, 0

```
1 .data
2 a: .word 3
3 b: .word 2
4 x: .word 0
5 .text
6 .globl main
7     la t0, a
8     lw a0, 0(t0)
9     la t1, b
10    lw a1, 0(t1)
11    jal ra, EqSolver
12    la t2, x
13    sw a0, 0(t2)
14    li a7, 10
15    ecall
16 EqSolver:
17     add t3, a0, a1
18     sub t4, a0, a1
19     mul a0, t3, t4
20     jalr zero, ra, 0
```

Output:

0x0	0x10000297	auipc x5 65536	la t0, a
0x4	0x00028293	addi x5 x5 0	la t0, a
0x8	0x0002A503	lw x10 0(x5)	lw a0, 0(t0)
0xc	0x10000317	auipc x6 65536	la t1, b
0x10	0xFF830313	addi x6 x6 -8	la t1, b
0x14	0x00032583	lw x11 0(x6)	lw a1, 0(t1)
0x18	0x018000EF	jal x1 24	jal ra, EqSolver
0x1c	0x10000397	auipc x7 65536	la t2, x
0x20	0xFEC38393	addi x7 x7 -20	la t2, x
0x20	0xFEC38393	addi x7 x7 -20	la t2, x
0x24	0x00A3A023	sw x10 0(x7)	sw a0, 0(t2)
0x28	0x00A00893	addi x17 x0 10	li a7, 10
0x2c	0x00000073	ecall	ecall
0x30	0x00B50E33	add x28 x10 x11	add t3, a0, a1
0x34	0x40B50EB3	sub x29 x10 x11	sub t4, a0, a1
0x38	0x03DE0533	mul x10 x28 x29	mul a0, t3, t4
0x3c	0x00008067	jalr x0 x1 0	jalr zero, ra, 0

a0 (x10)	5	t3 (x28)	5
a1 (x11)	2	t4 (x29)	1

Task 4:

Description:

In this task we are required to find area and perimeter of the square so we know the formula for

area=side*side and for perimeter=4*side so here we have to initialize side area and perimeter we initialize side with some value and for storing the area and perimeter we have taken it as 0 in memory so then we load the side in t0 register then load the value 5 and then jump to area finder where that formula is applied and then back to the main where we store that result in memory at offset 0 then same for perimeter we performed .

Code:

```
.data

side:   .word 5

area:   .word 0

perimeter: .word 0


.text


.globl main

    la t0, side

    lw a0, 0(t0)

    jal ra, AreaFinder

    la t1, area

    sw a0, 0(t1)


    lw a0, 0(t0)

    jal ra, PerimeterFinder

    la t2, perimeter

    sw a0, 0(t2)

    li a7, 10

    ecall
```

AreaFinder:

```
mul a0, a0, a0
```

```
jalr zero, ra, 0
```

PerimeterFinder:

```
li t3, 4
```

```
mul a0, a0, t3
```

```
jalr zero, ra, 0
```

```
1 .data
2 side:      .word 5
3 area:      .word 0
4 perimeter: .word 0
5
6 .text
7 .globl main
8     la t0, side
9     lw a0, 0(t0)
10    jal ra, AreaFinder
11    la t1, area
12    sw a0, 0(t1)
13
14    lw a0, 0(t0)
15    jal ra, PerimeterFinder
16    la t2, perimeter
17    sw a0, 0(t2)
18    li a7, 10
19    ecall
20
21 AreaFinder:
22     mul a0, a0, a0
23     jalr zero, ra, 0
24
25 PerimeterFinder:
26     li t3, 4
27     mul a0, a0, t3
28     jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code								
0x0	0x10000297	auipc x5 65536	la t0, side								
0x4	0x00028293	addi x5 x5 0	la t0, side								
0x8	0x0002A503	lw x10 0(x5)	lw a0, 0(t0)								
0xc	0x02C000EF	jal x1 44	jal ra, AreaFinder								
0x10	0x10000317	auipc x6 65536	la t1, area								
0x14	0xFF430313	addi x6 x6 -12	la t1, area								
0x18	0x00A32023	sw x10 0(x6)	sw a0, 0(t1)								
0x1c	0x0002A503	lw x10 0(x5)	lw a0, 0(t0)								
0x20	0x020000EF	jal x1 32	jal ra, PerimeterFinder								
0x24	0x10000397	auipc x7 65536	la t2, perimeter								
0x2c	0x00A3A023	sw x10 0(x7)	sw a0, 0(t2)								
0x30	0x00A00893	addi x17 x0 10	li a7, 10								
0x34	0x00000073	ecall	ecall								
0x38	0x02A50533	mul x10 x10 x10	mul a0, a0, a0								
0x3c	0x00008067	jalr x0 x1 0	jalr zero, ra, 0								
0x40	0x00400E13	addi x28 x0 4	li t3, 4								
0x44	0x03C50533	mul x10 x10 x28	mul a0, a0, t3								
0x48	0x00008067	jalr x0 x1 0	jalr zero, ra, 0								
<table> <tr> <td>a0 (x10)</td><td>5</td><td>a0 (x10)</td><td>25</td></tr> <tr> <td>a0 (x10)</td><td>5</td><td>a0 (x10)</td><td>20</td></tr> </table>				a0 (x10)	5	a0 (x10)	25	a0 (x10)	5	a0 (x10)	20
a0 (x10)	5	a0 (x10)	25								
a0 (x10)	5	a0 (x10)	20								

Task 5:

Description:

In this task firstly we initialize x by 2 and for saving result we initialize y as 0 in the memory then in main we load the x and then jump to the eqsolver function where the operations that is given is performed and then return to the main where we store the y in memory and then call to exit program.

Code:

```
..data
```

x: .word 2

y: .word 0

.text

.globl main

la t0, x

lw a0, 0(t0)

jal ra, EqSolver

la t1, y

sw a0, 0(t1)

li a7, 10

ecall

EqSolver:

mul t0, a0, a0 # t0 = x^2

mul t1, t0, a0 # t1 = x^3

li t2, 5 # t2 = 5

mul t3, t0, t2 # t3 = 5 * x^2

li t2, 6 # t2 = 6

mul t4, a0, t2 # t4 = 6 * x

```
add t5, t1, t3    # t5 = x^3 + 5x^2

add a0, t5, t4    # a0 = (x^3 + 5x^2) + 6x

jalr zero, ra, 0
```

```
1 .data
2 x: .word 2
3 y: .word 0
4
5 .text
6 .globl main
7     la t0, x
8     lw a0, 0(t0)
9
10    jal ra, EqSolver
11    la t1, y
12    sw a0, 0(t1)
13
14    li a7, 10
15    ecall
16
17 EqSolver:
18
19     mul t0, a0, a0    # t0 = x^2
20     mul t1, t0, a0    # t1 = x^3
21     li t2, 5         # t2 = 5
22     mul t3, t0, t2    # t3 = 5 * x^2
23     li t2, 6         # t2 = 6
24     mul t4, a0, t2    # t4 = 6 * x
25     add t5, t1, t3    # t5 = x^3 + 5x^2
26     add a0, t5, t4    # a0 = (x^3 + 5x^2) + 6x
27     jalr zero, ra, 0
```

Output:

a0 (x10)	2	t0 (x5)	4
t1 (x6)	8	t2 (x7)	5

t3 (x28)	20	t2 (x7)	6
t4 (x29)	12	t5 (x30)	28
a0 (x10)	40	t0 (x5)	1600
t4 (x29)	240		

Task 6:

Description:

In this task the if else condition is given firstly we initialize the n and k with some value and also y for storing result then in main we load the values that are the arguments of functions in a0 and a1 registers then we check for conditions and call the functions such as callf1,callf2,callf3 then they jump to the f1 f2 f3 where they perform the desired operations and then at the end return to the main program and then exit it.

Code:

```
.data

n: .word 4

k: .word 1

y: .word 0

.text

.globl main

    la t0, n

    lw a0, 0(t0)
```

```
la t1, k

lw a1, 0(t1)

li t2, 1

beq a1, t2, call_F1

bgt a0, a1, call_F2

j call_F3

call_F1:

jal ra, F1

j store_result

call_F2:

jal ra, F2

j store_result

call_F3:

jal ra, F3

j store_result

store_result:

la t3, y

sw a0, 0(t3)

li a7, 10

ecall

F1:
```

```
mul t0, a0, a0 # t0 = x^2  
  
mul t1, a1, a1 # t1 = z^2  
  
add a0, t0, t1 # a0 = x^2 + z^2  
  
jalr zero, ra, 0
```

F2:

```
mul t0, a0, a0 # t0 = z^2  
  
mul a0, t0, a0 # a0 = z^3  
  
jalr zero, ra, 0
```

F3:

```
addi a0, a0, 5 # a0 = w + 5  
  
jalr zero, ra, 0
```

```
1 .data
2 n: .word 4
3 k: .word 1
4 y: .word 0
5 .text
6 .globl main
7     la t0, n
8     lw a0, 0(t0)
9     la t1, k
10    lw a1, 0(t1)
11    li t2, 1
12    beq a1, t2, call_F1
13    bgt a0, a1, call_F2
14    j call_F3
15 call_F1:
16     jal ra, F1
17     j store_result
18 call_F2:
19     jal ra, F2
20     j store_result
21
22 call_F3:
23     jal ra, F3
24     j store_result
25 store_result:
26     la t3, y
27     sw a0, 0(t3)
28
29     li a7, 10
30     ecall
31 F1:
32     mul t0, a0, a0 # t0 = x^2
33     mul t1, a1, a1 # t1 = z^2
34     add a0, t0, t1 # a0 = x^2 + z^2
35     jalr zero, ra, 0
36 F2:
37     mul t0, a0, a0 # t0 = z^2
38     mul a0, t0, a0 # a0 = z^3
39     jalr zero, ra, 0
40 F3:
41     addi a0, a0, 5 # a0 = w + 5
42     jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	la t0, n
0x4	0x00028293	addi x5 x5 0	la t0, n
0x8	0x0002A503	lw x10 0(x5)	lw a0, 0(t0)
0xc	0x10000317	auipc x6 65536	la t1, k
0x10	0xFF830313	addi x6 x6 -8	la t1, k
0x14	0x00032583	lw x11 0(x6)	lw a1, 0(t1)
0x18	0x00100393	addi x7 x0 1	li t2, 1
0x1c	0x00758663	beq x11 x7 12	beq a1, t2, call_F1
0x20	0x00A5C863	blt x11 x10 16	bgt a0, a1, call_F2
0x24	0x0140006F	jal x0 20	j call_F3
0x28	0x02C000EF	jal x1 44	jal ra, F1
0x2c	0x0140006F	jal x0 20	j store_result
0x30	0x034000EF	jal x1 52	jal ra, F2
0x34	0x00C0006F	jal x0 12	j store_result
0x38	0x038000EF	jal x1 56	jal ra, F3
0x3c	0x0040006F	jal x0 4	j store_result
0x2c	0x0140006F	jal x0 20	j store_result
0x30	0x034000EF	jal x1 52	jal ra, F2
0x34	0x00C0006F	jal x0 12	j store_result
0x38	0x038000EF	jal x1 56	jal ra, F3
0x3c	0x0040006F	jal x0 4	j store_result
0x40	0x10000E17	auipc x28 65536	la t3, y
0x44	0xFC8E0E13	addi x28 x28 -56	la t3, y
0x48	0x00AE2023	sw x10 0(x28)	sw a0, 0(t3)
0x4c	0x00A00893	addi x17 x0 10	li a7, 10
0x50	0x00000073	ecall	ecall
0x54	0x02A502B3	mul x5 x10 x10	mul t0, a0, a0 # t0 = x^2
0x58	0x02B58333	mul x6 x11 x11	mul t1, a1, a1 # t1 = z^2
0x5c	0x00628533	add x10 x5 x6	add a0, t0, t1 # a0 = x^2 + z^2
0x60	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x64	0x02A502B3	mul x5 x10 x10	mul t0, a0, a0 # t0 = x^2
0x68	0x02A28533	mul x10 x5 x10	mul a0, t0, a0 # a0 = z^3
0x6c	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x70	0x00550513	addi x10 x10 5	addi a0, a0, 5 # a0 = w + 5
0x74	0x00008067	jalr x0 x1 0	jalr zero, ra, 0

ra (x1)	44
sp (x2)	2147483632
gp (x3)	268435456
tp (x4)	0
t0 (x5)	16
t1 (x6)	1
t2 (x7)	1
s0 (x8)	0
s1 (x9)	0
a0 (x10)	17
a1	1
a7 (x17)	10

Task 7:

Description:

In this task we firstly initialize the function arguments store some value in them and y is for saving the result in memory then we load n and k after that we started to call the functions that satisfy the condition and then in the called function we jump to the function and perform the desired operation on n then store it into the result when returned to the main and store it to the memory.

Code:

.data

```
n: .word 3

k: .word 3

y: .word 0

.text

.globl main

    la t0, n

    lw a0, 0(t0)

    la t1, k

    lw a1, 0(t1)

    beq a1, zero, call_fun1

    beq a0, a1, call_fun2

    j call_fun3

call_fun1:

    jal ra, fun1

    j result

call_fun2:

    jal ra, fun2

    j result

call_fun3:

    jal ra, fun3

    j result

result:

    la t2, y
```

```
sw a0, 0(t2)
```

```
li a7, 10
```

```
ecall
```

```
fun1:
```

```
mul a0, a0, a0 # a0 = x * x
```

```
jalr zero, ra, 0
```

```
fun2:
```

```
mul t0, a0, a0 # t0 = z^2
```

```
mul a0, t0, a0 # a0 = z^3
```

```
jalr zero, ra, 0
```

```
fun3:
```

```
addi a0, a0, 1 # a0 = w + 1
```

```
jalr zero, ra, 0
```

```
1 .data
2 n: .word 3
3 k: .word 3
4 y: .word 0
5 .text
6 .globl main
7     la t0, n
8     lw a0, 0(t0)
9     la t1, k
10    lw a1, 0(t1)
11    beq a1, zero, call_fun1
12    beq a0, a1, call_fun2
13    j call_fun3
14 call_fun1:
15     jal ra, fun1
16     j result
17 call_fun2:
18     jal ra, fun2
19     j result
20 call_fun3:
21     jal ra, fun3
22     j result
23 result:
24     la t2, y
25     sw a0, 0(t2)
26     li a7, 10
27     ecall
28 fun1:
29     mul a0, a0, a0 # a0 = x * x
30     jalr zero, ra, 0
31
32 fun2:
33     mul t0, a0, a0 # t0 = z^2
34     mul a0, t0, a0 # a0 = z^3
35     jalr zero, ra, 0
36
37 fun3:
38     addi a0, a0, 1 # a0 = w + 1
39     jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	la t0, n
0x4	0x00028293	addi x5 x5 0	la t0, n
0x8	0x0002A503	lw x10 0(x5)	lw a0, 0(t0)
0xc	0x10000317	auipc x6 65536	la t1, k
0x10	0xFF830313	addi x6 x6 -8	la t1, k
0x14	0x00032583	lw x11 0(x6)	lw a1, 0(t1)
0x18	0x00058663	beq x11 x0 12	beq a1, zero, call_fun1
0x1c	0x00B50863	beq x10 x11 16	beq a0, a1, call_fun2
0x20	0x0140006F	jal x0 20	j call_fun3
0x24	0x02C000EF	jal x1 44	jal ra, fun1
0x28	0x0140006F	jal x0 20	j result
0x2c	0x02C000EF	jal x1 44	jal ra, fun2
0x30	0x00C0006F	jal x0 12	j result
0x34	0x030000EF	jal x1 48	jal ra, fun3
0x38	0x0040006F	jal x0 4	j result
0x3c	0x10000397	auipc x7 65536	la t2, y
0x40	0xFCC38393	addi x7 x7 -52	la t2, y
0x44	0x00A3A023	sw x10 0(x7)	sw a0, 0(t2)
0x48	0x00A00893	addi x17 x0 10	li a7, 10
0x4c	0x00000073	ecall	ecall
0x50	0x02A50533	mul x10 x10 x10	mul a0, a0, a0 # a0 = x * x
0x54	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x58	0x02A502B3	mul x5 x10 x10	mul t0, a0, a0 # t0 = z^2
0x5c	0x02A28533	mul x10 x5 x10	mul a0, t0, a0 # a0 = z^3
0x60	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x64	0x00150513	addi x10 x10 1	addi a0, a0, 1 # a0 = w + 1
0x68	0x00008067	jalr x0 x1 0	jalr zero, ra, 0

a0 (x10)	3	a1 (x11)	3
t0 (x5)	9	a0 (x10)	27

Task 8(non-recursive):

Description:

In this task without recursion we are running for loop and by using temporary registers nth Fibonacci number calculated. In the main function load the n value from memory then store it in a0 register then call by fib then there we set the numbers in t0 t1 t2 then check the base condition if n==0 then return 0 if not then run from 2 to n counter where the operations performed then after it return b then returned to main func and store result in y.

Code:

```
.data

n: .word 6

y: .word 0


.text

.globl main

    la t0, n

    lw a0, 0(t0)

    jal ra, fib

    la t1, y

    sw a0, 0(t1)

    li a7, 10

    ecall

fib:

    li t0, 0    # t0 = a = 0

    li t1, 1    # t1 = b = 1

    beq a0, zero, return_a
```

```
li t2, 2    # t2 = i = 2

fib_loop:

    bgt t2, a0, loop_end # (i > n)

    add t3, t0, t1 # c = a + b

    mv t0, t1    # a = b

    mv t1, t3    # b = c

    addi t2, t2, 1 # i++

    j fib_loop

loop_end:

    mv a0, t1    # b

    jalr zero, ra, 0

return_a:

    mv a0, t0    # a(0)

    jalr zero, ra, 0
```

```
1 .data
2 n: .word 6
3 y: .word 0
4
5 .text
6 .globl main
7     la t0, n
8     lw a0, 0(t0)
9     jal ra, fib
10    la t1, y
11    sw a0, 0(t1)
12    li a7, 10
13    ecall
14 fib:
15     li t0, 0      # t0 = a = 0
16     li t1, 1      # t1 = b = 1
17     beq a0, zero, return_a
18     li t2, 2      # t2 = i = 2
19 fib_loop:
20     bgt t2, a0, loop_end # (i > n)
21     add t3, t0, t1 # c = a + b
22     mv  t0, t1     # a = b
23     mv  t1, t3     # b = c
24     addi t2, t2, 1 # i++
25     j  fib_loop
26 loop_end:
27     mv a0, t1      # b
28     jalr zero, ra, 0
29 return_a:
30     mv a0, t0      # a(0)
31     jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	la t0, n
0x4	0x00028293	addi x5 x5 0	la t0, n
0x8	0x0002A503	lw x10 0(x5)	lw a0, 0(t0)
0xc	0x018000EF	jal x1 24	jal ra, fib
0x10	0x10000317	auipc x6 65536	la t1, y
0x14	0xFF430313	addi x6 x6 -12	la t1, y
0x18	0x00A32023	sw x10 0(x6)	sw a0, 0(t1)
0x1c	0x00A00893	addi x17 x0 10	li a7, 10
0x20	0x00000073	ecall	ecall
0x24	0x00000293	addi x5 x0 0	li t0, 0 # t0 = a = 0
0x28	0x00100313	addi x6 x0 1	li t1, 1 # t1 = b = 1
0x2c	0x02050463	beq x10 x0 40	beq a0, zero, return_a
0x30	0x00200393	addi x7 x0 2	li t2, 2 # t2 = i = 2
0x34	0x00754C63	blt x10 x7 24	bgt t2, a0, loop_end # (i > n)
0x38	0x00628E33	add x28 x5 x6	add t3, t0, t1 # c = a + b
0x3c	0x00030293	addi x5 x6 0	mv t0, t1 # a = b
0x40	0x000E0313	addi x6 x28 0	mv t1, t3 # b = c
0x44	0x00138393	addi x7 x7 1	addi t2, t2, 1 # i++
0x48	0xFEDFF06F	jal x0 -20	j fib_loop
0x4c	0x00030513	addi x10 x6 0	mv a0, t1 # b
0x50	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x54	0x00028513	addi x10 x5 0	mv a0, t0 # a(0)
0x58	0x00008067	jalr x0 x1 0	jalr zero, ra, 0

a0 (x10)	6	t0 (x5)	0
t2 (x7)	2	t1 (x6)	1
t0 (x5)	1	t3 (x28)	1
t1 (x6)	1		



Task 8(recursive):

Description:

In this task the recursive fibonacci is given in we check the base case $n==0$ (0) or $n==1$ (1) then recursive case return $\text{fib}(n-1) + \text{fib}(n-2)$. We initialize n by 6 and storing result in y then we load in main in argument register $a0$ as n then jump to fib we check the cases by using stack and then at the end call exit when return to main.

Code:

```
1 .data
2 n: .word 6
3 y: .word 0
4
5 .text
6 .globl main
7     la t0, n
8     lw a0, 0(t0)    # a0 = n = 6
9     jal ra, fib
10    la t1, y
11    sw a0, 0(t1)
12    li a7, 10
13    ecall
14 fib:
15     beq a0, zero, return_zero # (n == 0) return 0
16     li t0, 1 #if (n == 1) return 1
17     beq a0, t0, return_one
18     addi sp, sp, -12 # Recursive Case: fib(n-1) + fib(n-2)
19     sw ra, 8(sp)
20     sw a0, 4(sp)
21     addi a0, a0, -1 # a0 = n - 1
22     jal ra, fib
23     sw a0, 0(sp)
24     lw a0, 4(sp)
25     addi a0, a0, -2 # a0 = n - 2
26     jal ra, fib
27     lw t0, 0(sp)
28     add a0, t0, a0 # a0 = fib(n-1) + fib(n-2)
29     lw ra, 8(sp)
30     addi sp, sp, 12
31     jalr zero, ra, 0 |
32 return_zero:
33     li a0, 0
34     jalr zero, ra, 0
35
36 return_one:
37     li a0, 1
38     jalr zero, ra, 0
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	la t0, n
0x4	0x00028293	addi x5 x5 0	la t0, n
0x8	0x0002A503	lw x10 0(x5)	lw a0, 0(t0) # a0 = n = 6
0xc	0x018000EF	jal x1 24	jal ra, fib
0x10	0x10000317	auipc x6 65536	la t1, y
0x14	0xFF430313	addi x6 x6 -12	la t1, y
0x18	0x00A32023	sw x10 0(x6)	sw a0, 0(t1)
0x1c	0x00A00893	addi x17 x0 10	li a7, 10
0x20	0x00000073	ecall	ecall
0x24	0x04050263	beq x10 x0 68	beq a0, zero, return_zero # (n == 0) return 0
0x28	0x00100293	addi x5 x0 1	li t0, 1 #if (n == 1) return 1
0x2c	0x04550263	beq x10 x5 68	beq a0, t0, return_one
0x30	0xFF410113	addi x2 x2 -12	addi sp, sp, -12 # Recursive Case: fib(n-1) + fib(n-2)
0x34	0x00112423	sw x1 8(x2)	sw ra, 8(sp)
0x38	0x00A12223	sw x10 4(x2)	sw a0, 4(sp)
0x38	0x00A12223	sw x10 4(x2)	sw a0, 4(sp)
0x3c	0xFF50513	addi x10 x10 -1	addi a0, a0, -1 # a0 = n - 1
0x40	0xFE5FF0EF	jal x1 -28	jal ra, fib
0x44	0x00A12023	sw x10 0(x2)	sw a0, 0(sp)
0x48	0x00412503	lw x10 4(x2)	lw a0, 4(sp)
0x4c	0xFFE50513	addi x10 x10 -2	addi a0, a0, -2 # a0 = n - 2
0x50	0xFD5FF0EF	jal x1 -44	jal ra, fib
0x54	0x00012283	lw x5 0(x2)	lw t0, 0(sp)
0x58	0x00A28533	add x10 x5 x10	add a0, t0, a0 # a0 = fib(n-1) + fib(n-2)
0x5c	0x00812083	lw x1 8(x2)	lw ra, 8(sp)
0x60	0x00C10113	addi x2 x2 12	addi sp, sp, 12
0x64	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x68	0x00000513	addi x10 x0 0	li a0, 0
0x6c	0x00008067	jalr x0 x1 0	jalr zero, ra, 0
0x70	0x00100513	addi x10 x0 1	li a0, 1
0x74	0x00008067	jalr x0 x1 0	jalr zero, ra, 0

a0
(x10)

6

t0
(x5)

1

t0 (x5)	1	a0 (x10)	5
a0 (x10)	4		